

ACTIVIDAD 1. API, JSON o Comunicación Cliente-Servidor

Clasificación de Conceptos

A) API: API, Endpoint, GET, POST, Servicio Web, Recurso, REST, PUT, DELETE, URL, HTTPS, status code, Header, Body, Flask.

B) JSON: JSON, Objeto, Propiedad, Valor, Array, Number, String, Boolean, Null, llaves {}.

C) Comunicación Cliente-Servidor: Cliente, Servidor, HTTP, Request, Response, Postman, HTML, CSS, PYTHON.

Cuestionario y Reto JSON

1. ¿Qué relación existe entre una API, HTTP y JSON?

La API define las reglas y rutas (endpoints) para la comunicación. HTTP es el protocolo de transferencia utilizado para enviar las peticiones (Request) y respuestas (Response), y JSON es el formato ligero de datos en el que se estructuran los mensajes intercambiados entre el cliente y el servidor.

2. ¿Cuál es una propiedad del siguiente objeto JSON?"nombre", "edad" o "carrera".

3. ¿Cuál es un valor de texto y cuál es un valor numérico?

Valor de texto (String): "Juan" (o "Programación").

Valor numérico (Number): 18.

4. ¿Qué representan las llaves {} en JSON?

Representan la delimitación de un objeto JSON (un conjunto de pares clave-valor).

5. ¿Por qué JSON puede utilizarse para intercambiar información entre cliente y servidor?

Porque es un formato estándar de texto plano, independiente del lenguaje de programación, liviano de transferir y fácil de interpretar tanto para humanos como para los sistemas informáticos.

Respuestas de la Actividad 2

6. Agrega una nueva ruta /redes donde salude:(Incluida en el código de arriba).

7. ¿Cuál es el endpoint /alumnos?

Es la ruta específica URL ([http://127.0.0.1:5000/alumnos/configurada en el servidor para consultar el recurso que devuelve el listado de los alumnos registrados.

8. ¿Qué método HTTP estamos utilizando?

Método GET.

9. ¿Qué información devuelve el endpoint /alumnos y el endpoint de /redes?

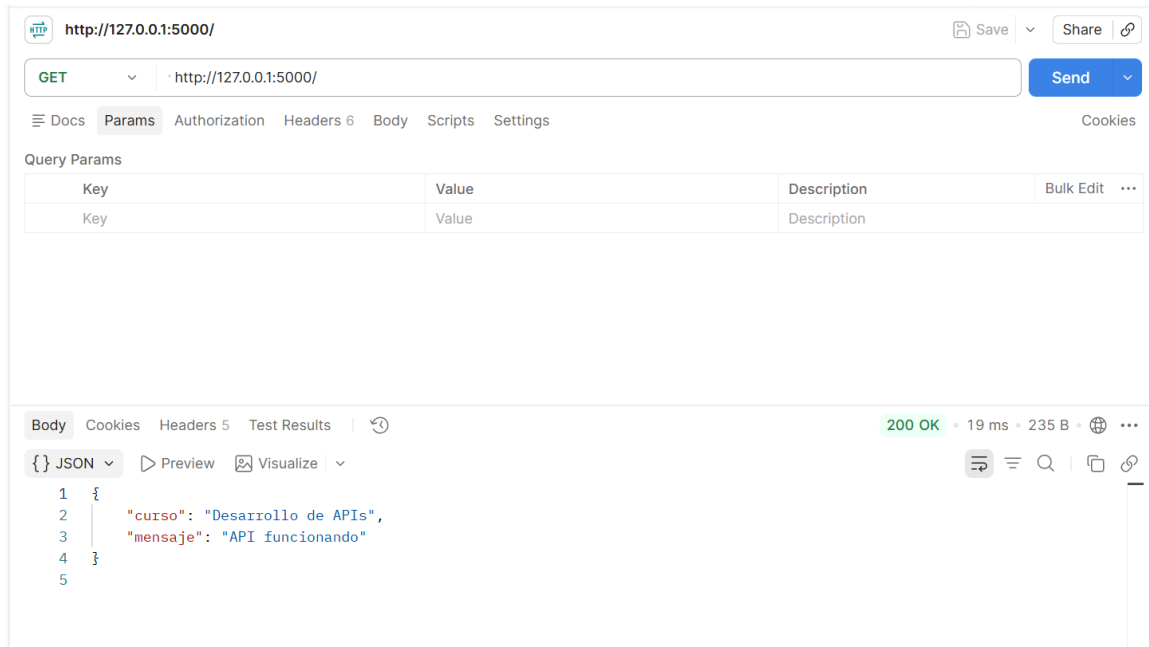
/alumnos: Devuelve una lista (arreglo JSON) con objetos que contienen el id y nombre de cada alumno.

/redes: Devuelve un objeto JSON con un mensaje de saludo.

```
C:\Users\Ricardo\Desktop\CuartoCuatri\Programacion_Redes\DesafiosU2\Api_flask>python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 137-589-695
127.0.0.1 - - [02/Oct/2026 12:47:15] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [02/Oct/2026 12:47:15] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [02/Oct/2026 12:47:32] "GET /alumnos HTTP/1.1" 200 -
* Detected change in 'C:\\Users\\Ricardo\\Desktop\\CuartoCuatri\\Programacion_Redes\\DesafiosU2\\Api_flask\\app.py', re
loading
* Restarting with stat
* Debugger is active!
* Debugger PIN: 137-589-695
127.0.0.1 - - [02/Oct/2026 12:56:22] "GET /alumnos HTTP/1.1" 200 -
127.0.0.1 - - [02/Oct/2026 12:59:50] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [02/Oct/2026 13:00:11] "GET /alumnos HTTP/1.1" 200 -
127.0.0.1 - - [02/Oct/2026 13:00:11] "GET /alumnos HTTP/1.1" 200 -
127.0.0.1 - - [02/Oct/2026 13:00:14] "GET /alumnos HTTP/1.1" 200 -
```

The screenshot shows a web browser interface for testing HTTP requests. The address bar shows the URL `http://127.0.0.1:5000/alumnos`. The method is set to `GET`. The response status is `200 OK` with a response time of `7 ms` and a size of `257 B`. The response body is displayed in JSON format:

```
[{"id": 1, "nombre": "Ana"}, {"id": 2, "nombre": "Carlos"}]
```



elemento
funcion

GET
Solicita la informacion de un servidor

URL
Direcccion del servidor

SEND
Envia una peticion

RESPONSE
área donde se despliega la respuesta
devuelta por el servidor tras procesar la petición.

STATUS CODE
regresado del esttus del servidor
Codigo

BODY
Cuerpo donde aparece las respuestas

HEADERS
HTTP
Encabezados

Reto Final

Explica con tus propias palabras: ¿qué ventaja tiene Postman respecto a probar una API solamente desde el navegador?

Probar una API desde el navegador limita las pruebas casi exclusivamente a peticiones de lectura (GET). Postman ofrece herramientas avanzadas esenciales para el desarrollo, tales como:

Enviar todo tipo de métodos HTTP (POST, PUT, DELETE, PATCH, etc.).

Personalizar y agregar fácilmente Headers, parámetros de consulta y autenticaciones.

Manipular e inspeccionar directamente el Body en JSON u otros formatos para pruebas avanzadas.

Analizar a detalle los Status Codes, tiempos de respuesta y encabezados devueltos por el servidor.